Now, eventually we are likely to need third-party JavaScript modules added to our system, and importmaps-rails provides a way to do that. There is a service, called jspm.org, that is a content delivery network (CDN) for node packages via NPM, with an API specifically to allow import maps to reference those node modules.

The importmap-rails gem has a command-line tool that allows you to specify the name of an npm package. The command line queries jspm.org for that package and its dependencies, and adds those entries to the config/importmap.rb file with the URL of those packages from jspm.

For example, in our project we have the form-request-submit-polyfill module that we used in Chapter 3, *Stimulus* to submit a form programatically. If we want to use that in our import map context, we can do that from the command line:

```
$ bin/importmap pin form-request-submit-polyfill
```

The pin command goes back to jspm, determines the URL of the module, and adds it to the config file like so:

```
pin(
  "form-request-submit-polyfill",
  to: ("https://ga.jspm.io/npm:form-request-submit-polyfill@2.0.0/" +
      "form-request-submit-polyfill.js")
)
```

If there are dependent modules, those are also added.

Now when we send import maps to the browser, form-request-submit-polyfill is included and downloaded from the jspm CDN. If you'd rather it be managed locally, adding --download or -d will download to the vendor directory it so that it will be served locally from your server. You can add the preload option directly to the config file if you want it preloaded.

You can specify a specific version by using @syntax as in bin/importmap pin @hotwired/stimulus@3.0.0, and @latest will work for modules that are set up to handle it. Pinning a package that has already been pinned will update it to the most